



Bill Gram

Shopkeeper Billing & Inventory Management — Full Project Documentation

ELECTRON 28

REACT 19

NODE.JS + EXPRESS

SQLITE + PRISMA

WINDOWS DESKTOP



Bill Gram — Shopkeeper Billing & Inventory Management

A full-featured **desktop billing and inventory management application** for Indian shopkeepers and small businesses. Built as a cross-platform **Electron app** with a local React frontend, Node.js backend, and SQLite database — works completely **offline**.



Table of Contents

- Overview
- Features
- Tech Stack
- Project Structure
- Prerequisites
- Getting Started
- Environment Variables
- Database
- Building for Production
- Key Modules
- Scripts Reference
- License

Overview

Bill Gram is a **Windows desktop application** that helps shopkeepers manage their entire business from one place:

- Create and print **GST-compliant purchase & sales invoices**
- Manage **product inventory** with batch tracking, barcodes, and MRP
- Handle **suppliers and customers** with full contact and GST-IN details
- Generate **PDF and Excel reports** (Purchase, Sales, GSTR-1, GSTR-2, B2B, B2C)
- **Backup and restore** your database to Google Drive automatically

All data is stored locally on your machine — no subscription or internet required for core features.

Features

Authentication

- Secure user registration & login with **JWT-based sessions**
- Local credential store for offline login support
- Subscription expiry management

Product Inventory

- Rich product profiles: HSN/SAC code, weight, unit, brand, group, company
- **Batch management** per product (MFG date, EXP date, barcode, MRP, rates)
- GST configuration: rate, CESS, inclusive/exclusive toggle
- Product image gallery (main image + related images)
- Opening stock, purchase rate, sale rate, margin tracking
- Soft delete support

Master Data

Manage all your reference data in one place:

MASTER	DETAILS
Units	kg, pcs, ltr, box, etc.
Product Groups	Categories/classifications
Product Companies	Manufacturers with contact details
Suppliers	Name, phone, GST-IN, address
Customers	Company name, contact, GST-IN, area, type
Salesmen	Name, phone, area assignment
Areas	State, region, city, pincode
Vans / Vehicles	Fleet management for delivery
Bank Accounts	IFSC, GPay number, QR code

Purchase Module

- Create **purchase invoices** linked to a supplier
- Per-item tracking: actual qty, free qty, damaged qty, rate, scheme %, GST
- Invoice-level financials: gross → CESS/insurance → scheme → discount → tax → **final amount**
- Status tracking: Pending / Paid / Cancelled
- Automatic **stock update** on purchase via batch management
- Complete purchase history audit log

Sales Module

- Create **sales invoices** with customer, salesman, van, and area
- Per-item: quantity, rate, scheme %, GST, free qty, damaged qty
- Invoice financials: gross → CESS/insurance → scheme → discount → tax → **final amount**
- Status tracking: Pending / Paid / Cancelled / Delivered
- Sales history audit log per product/batch

Reports

Generate professional reports in **PDF** (via Puppeteer + EJS) and **Excel** (via ExcelJS):

- **Purchase Reports:** Summary, item-wise register
- **Sales Reports:** Summary, register, area-wise, salesman-wise
- **GST Reports:** GSTR-1, GSTR-2, B2B, B2C breakdowns

- Report history saved per user for re-download

Notifications

- In-app notification centre with types: `success`, `warning`, `error`, `info`, `create`, `update`, `delete`
- Per-section and per-page tagging
- Read/unread state tracking

User Profile

- Shop name, company name, address, phone
- Upload **company logo** and **signature** (embedded in invoice PDF)
- **UPI ID** displayed on printed invoices
- Notification and sound preferences

Backup & Restore

- **Google Drive** OAuth2 integration for cloud backup
- Auto (via `node-cron`) and manual backup triggers
- Backup history with status (`success` / `failed` / `skipped`), file size, Drive link
- One-click restore from a local or Drive backup file
- Local file backup to `Documents/ShopkeeperBackups/`

Tech Stack

LAYER	TECHNOLOGY	VERSION
Desktop Shell	Electron	28.x
Frontend	React + TypeScript	19.x
Build Tool	Vite	7.x
UI Components	shadcn/ui (Radix UI)	latest
Animations	Framer Motion	12.x
Charts	Recharts	3.x
State Management	Redux Toolkit	2.x
Forms	React Hook Form + Zod	latest
Styling	Tailwind CSS v4	4.x
HTTP Client	Axios	1.x
Backend	Node.js + Express	4.x
ORM	Prisma	5.x
Database	SQLite (local file)	—
Auth	bcryptjs + JWT	—
PDF Generation	Puppeteer + EJS templates	—
Excel Generation	ExcelJS	4.x
File Compression	Archiver / Unzipper	—
Cloud Backup	Google Drive API (googleapis)	—
Scheduled Jobs	node-cron	4.x
QR Codes	qrcode	1.x
Packaging	electron-builder (NSIS)	24.x

Project Structure

```

Bill_gram/
|
|
├── electron/                                # Electron main process
|
|   ├── main.js                            # App entry: spawns backend, creates window
|   └── preload.js                          # Secure IPC bridge to renderer
|
|
├── client/                                # React frontend (Vite + TypeScript)
|
|   └── src/
|
|       ├── pages/                          # Full-page route components
|       |
|       |   ├── Dashboard.tsx
|       |   ├── Login.tsx / Register.tsx
|       |   ├── ProductInventory.tsx / ProductForm.tsx
|       |   ├── MasterInventory.tsx
|       |   ├── Purchase.tsx / Sales.tsx
|       |   ├── Report.tsx
|       |   ├── Profile.tsx
|       |   └── Notifications.tsx

```

```

|   |   └─ Restore_Backup.tsx

|   └─ components/           # Reusable UI components

|   |   └─ common/           # Navbar, layout elements

|   |   └─ dashboard/        # Dashboard widgets

|   |   └─ product/           # Product-specific components

|   |   └─ purchase/          # Purchase invoice components

|   |   └─ sales/             # Sales invoice components

|   |   └─ master/            # Master data forms & tables

|   |   └─ report/            # Report UI components

|   |   └─ forms/             # Shared form components

|   |   └─ custom_ui/         # Custom UI elements

|   |   └─ ui/                # shadcn/ui base components

|   └─ contexts/             # AuthContext, ThemeProvider

|   └─ store/                 # Redux store & slices

|   └─ api/                   # Axios API call wrappers

|   └─ hooks/                 # Custom React hooks

```

```

|   └─ services/           # Business logic services

|   └─ types/              # TypeScript interfaces & types

|   └─ utils/              # Helper utilities

|

└─ server/                 # Node.js + Express backend

|   └─ src/

|       └─ server.js        # Express app entry point

|       └─ controllers/     # 18 resource controllers

|           └─ Auth/         └─ Product/

|           └─ Purchase/     └─ Sales/

|           └─ Customer/     └─ Supplier/

|           └─ Salesman/     └─ Area/

|           └─ Van/          └─ Unit/

|           └─ Product_Group/ └─ Product_Company/

|           └─ Account/      └─ Dashboard/

|           └─ Notification/  └─ Backup/

```



```
└─ installer.nsi # Custom NSIS installer script
```

✓ Prerequisites

Make sure you have the following installed:

- **Node.js** v18 or higher — nodejs.org
- **npm** v9 or higher (comes with Node.js)
- **Git** — git-scm.com

Getting Started

1. Clone the repository

```
git clone https://github.com/your-org/bill-gram.git
cd bill-gram
```

2. Install all dependencies

```
npm run install-all
```

This installs dependencies for the root, `client/`, and `server/` packages.

3. Set up environment variables

```
# Root .env
```

```
cp .env.example .env
```

Client .env

```
cp client/.env.example client/.env
```

Server .env

```
cp server/.env.example server/.env
```

See Environment Variables for details.

4. Set up the database

```
cd server
```

```
npm run prisma:migrate
```

5. Start in development mode

```
# From the root directory
```

```
npm run dev
```

This concurrently starts:

- **Backend** — Express server (via nodemon)
- **Frontend** — Vite dev server at <http://localhost:5173>
- **Electron** — Opens the desktop window once Vite is ready

Environment Variables

Root `.env`

```
# (usually empty – used for shared Electron env vars)
```

server/.env

```
DATABASE_URL="file:./data/shopkeeper.db"
```

```
JWT_SECRET="your-super-secret-jwt-key"
```

```
PORT=5000
```

Google Drive OAuth2 (for backup feature)

```
GOOGLE_CLIENT_ID="your-google-client-id"
```

```
GOOGLE_CLIENT_SECRET="your-google-client-secret"
```

```
GOOGLE_REDIRECT_URI="http://localhost:5000/api/backup/oauth/callback"
```

client/.env

```
VITE_API_URL="http://localhost:5000"
```

Database

Bill Gram uses **SQLite** via **Prisma ORM**. The database file is stored locally:

ENVIRONMENT	LOCATION
Development	<code>server/data/shopkeeper.db</code>
Production (Windows)	<code>%LOCALAPPDATA%\Shopkeeper\shopkeeper.db</code>

Key Models

MODEL	DESCRIPTION
<code>User</code>	App user with shop profile and subscription
<code>Product</code>	Product catalog with GST details
<code>Batch</code>	Per-product stock batches with barcode, MRP, rates
<code>PurchaseInvoice</code>	Purchase invoices from suppliers
<code>SalesInvoice</code>	Sales invoices to customers
<code>Customer</code>	Customer master with GST-IN
<code>Supplier</code>	Supplier master with GST-IN
<code>Salesman</code>	Salesman with area assignment
<code>Area</code>	Geographic areas (state/region/city)
<code>Van</code>	Delivery vehicles
<code>Account</code>	Bank accounts for payment
<code>Notification</code>	In-app notifications
<code>BackupHistory</code>	Backup/restore audit trail
<code>GoogleDriveToken</code>	Stored OAuth2 token per user

Prisma Commands

```
cd server
```

Generate Prisma client after schema changes

```
npm run prisma:generate
```

Run migrations (development)

```
npm run prisma:migrate
```

Push schema without migration (quick dev)

```
npm run prisma:push
```

Building for Production

1. Build all layers

```
npm run build
```

This runs:

- `client/` : TypeScript compile + Vite production build → `client/dist/`
- `server/` : no-op (plain Node.js, no compilation needed)

2. Package into Windows installer

```
npm run dist
```

This uses `electron-builder` to produce a Windows NSIS installer at `dist/`.

Targets:

- `x64` (64-bit) and `ia32` (32-bit) Windows executables
- One-click installer or custom installation directory option
- Creates desktop shortcut and Start Menu entry
- App data deleted on uninstall (configurable)

Output

```
dist/
└── Shopkeeper App Setup 1.0.0.exe  ← Windows installer
```

Key Modules

IPC Channels (Electron ↔ Renderer)

CHANNEL	DESCRIPTION
<code>get-database-location</code>	Returns path to <code>shopkeeper.db</code>
<code>backup-database</code>	Copies DB to <code>Documents/ShopkeeperBackups/</code>
<code>restore-database</code>	Restores DB from a backup file path
<code>open-database-folder</code>	Opens DB folder in Windows Explorer
<code>save-user-credential</code>	Saves user login credential locally
<code>get-user-credential</code>	Reads saved credential by email

API Endpoints Structure

POST /api/auth/register

POST /api/auth/login

GET /api/auth/me

GET /api/products

POST /api/products

PUT /api/products/:id

DELETE /api/products/:id

GET /api/purchases

POST /api/purchases

...

GET /api/sales

POST /api/sales

...

GET /api/reports/purchase

GET /api/reports/sales

GET /api/reports/gst

POST /api/backup/google-drive

POST /api/backup/restore

GET /api/backup/history

Scripts Reference

COMMAND	DESCRIPTION
<code>npm run dev</code>	Start all 3 processes (server + client + electron)
<code>npm run dev:client</code>	Start only the Vite frontend
<code>npm run dev:server</code>	Start only the Express backend
<code>npm run dev:electron</code>	Start only Electron (waits for Vite)
<code>npm run build</code>	Build client + server for production
<code>npm run dist</code>	Package into Windows installer
<code>npm run package</code>	Package without installer (directory)
<code>npm run clean</code>	Remove all build artifacts
<code>npm run install-all</code>	Install deps for root + client + server

License

This project is licensed under the terms found in LICENSE.

<div align="center">

<p>Built with ❤️ for Indian shopkeepers</p>

<p>

Electron · React · Node.js · Prisma
· SQLite

</p>

</div>